

F O U N D R Y

Datasets

Understanding how data is stored,
versioned, and managed in Palantir Foundry



What is a Dataset?

A wrapper around a collection of files stored in a backing file system, providing integrated support for permission management, schema management, version control, and updates over time.

Structured

Tabular data

Files in open-source formats like Parquet, with column metadata stored as a schema.

Unstructured

Files & media

Images, video, PDFs — no associated schema required.

Semi-Structured

XML / JSON

Possible to apply schemas, but better to infer tabular schema in downstream transformations.

Transactions

Transaction Lifecycle

OPEN

Files can be written to the dataset's backing file system

COMMITTED

Transaction finalized; written files appear in the latest dataset view

ABORTED

Transaction cancelled; written files are ignored



Git for Data

Dataset transactions are analogous to Git commits — each one is an atomic change to the contents of a dataset.

This enables Foundry's robust data versioning system.

Git Commit → Dataset Transaction

Branch → Dataset Branch

Transaction Types

SNAPSHOT

Replaces the entire dataset view with a new set of files. Simplest type; basis of batch pipelines.

APPEND

Adds new files to current view. Cannot overwrite existing files. Basis of incremental pipelines.

UPDATE

Adds new files and may overwrite contents of existing files in the current view.

DELETE

Removes file references from the dataset view. Does NOT delete the underlying file from the filesystem. Used for data retention.

Example: `SNAPSHOT(A,B) → APPEND(C) → UPDATE(A→A') → DELETE(B)` = View contains A' and C

Branches & Dataset Views

Branches

Branches enable collaboration by allowing multiple users to experiment with changes to a dataset simultaneously — much like branches in Git.

Dataset Views

Represents the effective file contents of a dataset for a branch at a point in time. Historical views are analogous to historical versions of the dataset.

How a View is Calculated

1 Start with an empty set of files

2 Begin at the latest SNAPSHOT transaction before the point in time

3 For each subsequent transaction:
APPEND/SNAPSHOT = add files; UPDATE = add + overwrite; DELETE = remove

Schemas

Metadata on a dataset view that defines how files should be interpreted — including parsing rules and column names/types.

Supported Field Types

BOOLEAN • BYTE • SHORT • INTEGER • LONG • FLOAT •
DOUBLE • DECIMAL • STRING • MAP • ARRAY • STRUCT •
BINARY • DATE • TIMESTAMP

File Formats

Parquet • Avro • Text (CSV, JSON variants)

Text format uses customMetadata for parsing configuration.



Note

There is no guarantee that files in a dataset conform to the specified schema. Mismatches cause errors in client applications. Schemas can change over time as dataset structure evolves.

Storage & Retention

Backing Filesystem

Files tracked in a dataset are NOT stored in Foundry itself. Instead, a mapping is maintained between each file's logical path in Foundry and its physical path in a backing file system.

The backing filesystem is specified by a base directory in a Hadoop FileSystem — typically a cloud storage provider.

Self-hosted

HDFS Cluster

Cloud Storage

Amazon S3 (most common)

Data Retention

DELETE transactions do not actually remove files from the backing filesystem — they only remove file references from the dataset view.

Retention Policies are used to physically remove data from old transactions that are no longer needed, helping to:

- Minimize storage costs
- Comply with data governance requirements

Key Takeaways

01

Datasets are file wrappers —

Providing schema, permissions, versioning, and lifecycle management.

02

Transactions drive change —

SNAPSHOT, APPEND, UPDATE, DELETE — each an atomic modification to dataset contents.

03

Branches enable collaboration —

Multiple users can experiment independently; views track state at any point in time.

04

Schemas are flexible —

Can change over time as datasets evolve; support 15+ field types across Parquet, Avro, and Text.

05

Storage is decoupled —

Files live in a backing filesystem (typically S3); DELETE only removes references until retention clears them.